GUIA PRÁTICO · DO BÁSICO AO AVANÇADO

```
kali@lab:~$ whoami
hacker-etico
kali@lab:~$ nmap -sV alvo.lab
22/tcp open ssh
80/tcp open http Apache 2.4.7
kali@lab:~$ ./aprender.sh _
```

Hacking ético

Do básico ao avançado: entenda como os ataques funcionam para saber defender

LABORATÓRIO · WEB · SENHAS · REDES · CARREIRA

GUIA PRÁTICO

Hacking ético

Do básico ao avançado: entenda como os ataques funcionam para saber defender

Hacking ético

Do básico ao avançado: entenda como os ataques funcionam para saber defender

Edição de 2026. Tudo neste livro se pratica em laboratório próprio, em plataformas de treino ou em testes autorizados. Acessar sistemas sem autorização é crime.

Este livro tem fins educativos e de formação profissional em segurança defensiva e testes autorizados.

Ilustrações e diagramas feitos para esta edição.

Sumário

Antes de começar 1

PARTE 1 · A MENTALIDADE

1 Como um hacker pensa 5

2 Uma breve história e o campo hoje 9

PARTE 2 · O LABORATÓRIO

3 Montando seu laboratório 14

4 As bases: redes e sistemas 18

PARTE 3 · AS FASES DE UM TESTE

5 As cinco fases de um teste 23

6 Reconhecimento e varredura 26

7 Exploração e pós-exploração 30

PARTE 4 · OS GRANDES DOMÍNIOS

8 Segurança de aplicações web 35

9 Senhas e autenticação 39

10 Redes sem fio e o fator humano 43

PARTE 5 · O OUTRO LADO E A CARREIRA

11 Pensar como quem defende 48

12 Seu caminho a partir daqui 52

Glossário 56

Antes de começar

Quase todo mundo que se interessa por hacking começou pela parte errada da pergunta. A dúvida parece ser "como eu invado?". A pergunta certa, a que separa quem fica bom de quem só copia comandos, é outra: **"por que isto funciona?"**

Entender *por que* uma senha fraca cai, *por que* um site deixa escapar dados de outro usuário, *por que* um programa trava e entrega o controle da máquina, é o que torna alguém capaz de encontrar falhas novas, e também de corrigi-las. Este livro ensina a pensar assim.

Hacking, no sentido original, é a curiosidade de descobrir como as coisas funcionam por dentro e fazê-las se comportar de formas que ninguém planejou. Essa curiosidade move tanto quem ataca quanto quem defende. A diferença entre os dois não está na técnica. Está na **autorização**.

A regra que vale mais que todas as técnicas

Este é um livro de **hacking ético**. Tudo o que você vai aprender se pratica em um destes três lugares, e em nenhum outro:

1. **No seu próprio laboratório**, com máquinas que você mesmo monta para serem atacadas (o Capítulo 3 ensina).
2. **Em plataformas feitas para treinar**, como TryHackMe, Hack The Box, PortSwigger Web Security Academy e o OWASP Juice

Shop, que existem justamente para você atacar à vontade, de forma legal.

3. **Em testes autorizados**, com permissão escrita do dono do sistema, que é o trabalho do pentester profissional.

> **ISTO NÃO É UM DETALHE BUROCRÁTICO**
>
> Acessar um sistema sem autorização é crime, mesmo que você não quebre nada e mesmo que a falha fosse "óbvia". No Brasil, a invasão de dispositivo informático é crime pelo artigo 154-A do Código Penal. Testar o site da escola, o Wi-Fi do vizinho ou a conta de alguém "só para ver" pode dar processo e antecedentes criminais. O alvo é sempre um sistema **seu** ou que você tem **permissão por escrito** para testar.

Por que insistir nisso logo na primeira página? Porque a indústria inteira de segurança é construída sobre essa linha. Um pentester e um criminoso usam as mesmas ferramentas; o que o primeiro tem, e o segundo não, é um contrato e uma conduta. Empresas contratam, pagam bem e confiam justamente em quem demonstra que entende essa diferença.

Como o livro está organizado

Parte	O que você aprende
1. A mentalidade	Como um hacker pensa, a história da área e o campo de trabalho hoje
2. O laboratório	Montar seu ambiente de testes e as bases de redes e sistemas
3. As fases de um teste	Reconhecimento, varredura, exploração e pós-exploração, em laboratório
4. Os grandes domínios	Web, senhas, redes sem fio e engenharia social
5. O outro lado e a carreira	Como tudo isso vira defesa, e como entrar na profissão

Cada capítulo traz um **caso real** que ficou famoso, uma parte técnica explicada do porquê, e um **laboratório** para você praticar com segurança.

> **PARA QUEM É ESTE LIVRO**
>
> Para quem está começando e quer um caminho honesto, do zero ao ponto de resolver desafios sozinho e entender o trabalho de um profissional. Não é preciso saber programar, mas vontade de mexer, errar e pesquisar é obrigatória. Hacking se aprende **fazendo**, não lendo.

PARTE 1

A mentalidade

CAPÍTULO 1

Como um hacker pensa

Pergunte a dez pessoas o que é um hacker e você vai ouvir sobre capuzes pretos e telas cheias de código verde. A realidade é menos cinematográfica e muito mais interessante: um hacker é alguém que olha para um sistema e, em vez de perguntar "o que isto faz?", pergunta "o que eu **consigo fazer** com isto que não era para eu conseguir?".

A pergunta que muda tudo

Toda tecnologia é construída a partir de suposições. O programador que fez um site supôs que o campo "idade" receberia um número de dois dígitos. Supôs que ninguém mudaria o endereço da página na mão. Supôs que o usuário só clicaria nos botões que ele desenhou. Cada suposição dessas é uma porta que o criador nem percebeu que deixou.

O hacker é quem testa as suposições. Ele digita um texto gigante no campo de idade. Ele muda o número na barra de endereço de `pedido=1032` para `pedido=1033` e vê se aparece o pedido de outra pessoa. Ele manda ao servidor uma requisição que nenhum botão da tela produziria.



O usuário comum anda pelo caminho pensado. O hacker testa as bordas.

Nada disso exige gênio. Exige um hábito mental: **nunca confiar que as coisas funcionam só do jeito pretendido**.

As qualidades que importam

Ferramentas se aprendem em um fim de semana. O que leva tempo, e o que os empregadores realmente procuram, são estas qualidades:

- **Curiosidade teimosa**: a disposição de passar horas entendendo por que algo se comporta de um jeito estranho.
- **Tolerância à frustração**: 95% das tentativas não dão em nada. Quem desiste na décima tentativa não chega na centésima, que funciona.
- **Pensamento sistemático**: testar uma coisa de cada vez, anotar o que mudou, não sair atirando.
- **Leitura cuidadosa**: a resposta quase sempre está numa mensagem de erro, num detalhe da documentação, num cabeçalho que ninguém olhou.

- **Humildade**: assumir que você não entendeu ainda, em vez de assumir que o sistema está errado.

> **O MÉTODO DE VERDADE**
>
> Observar → levantar uma hipótese → testar uma coisa só → ver o resultado → repetir. É o método científico aplicado a sistemas. Hacking é, no fundo, ciência experimental com computadores.

Os chapéus

A cultura da área separa os hackers por "chapéus", uma metáfora dos faroestes antigos:

Chapéu	Quem é
Branco (*white hat*)	Age com autorização, para defender. É o pentester, o pesquisador de segurança.
Preto (*black hat*)	Age sem autorização, para proveito próprio ou dano. É o criminoso.
Cinza (*gray hat*)	Age sem autorização, mas sem intenção de dano, por exemplo, achando e reportando falhas sem permissão. Mesmo "bem-intencionado", costuma ser ilegal.

Este livro é inteiro sobre o chapéu branco. Não por moralismo, mas por estratégia: é o único caminho que constrói uma carreira em vez de destruí-la.

O “hacker” original

A palavra nasceu nos anos 1960, no MIT, entre estudantes que adoravam desmontar e melhorar sistemas, de trens elétricos a computadores. Um “hack” era uma solução engenhosa e inesperada. Só muito depois, com os crimes digitais, a imprensa colou à palavra o sentido de “criminoso”. Entre quem é da área, o sentido antigo sobrevive: hackear é dominar um sistema a ponto de fazê-lo dançar conforme a sua música.

PARA FIXAR

- [ ] Entendi que hacking é testar as suposições de quem construiu o sistema.
- [ ] Sei as qualidades que importam mais que as ferramentas.
- [ ] Conheço a diferença entre chapéu branco, preto e cinza.

CAPÍTULO 2

Uma breve história e o campo hoje

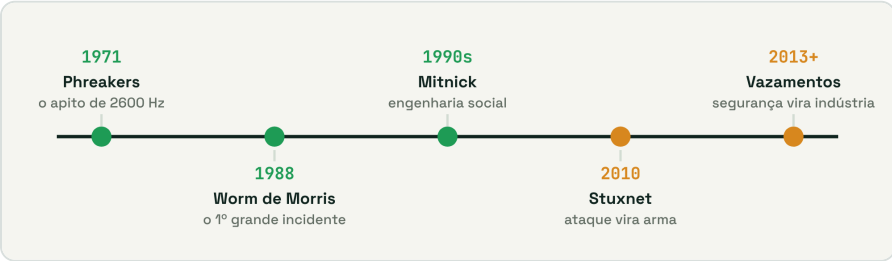
Conhecer de onde a área veio ajuda a entender por que ela é como é, e mostra que quase toda técnica "nova" é a repetição de uma ideia antiga num contexto novo.

Os marcos que vale conhecer

Os phreakers (anos 1960-70). Antes dos computadores pessoais, os primeiros hackers brincavam com o sistema telefônico. Descobriram que um tom de 2600 Hz, produzido por acaso por um apito de brinde de uma caixa de cereal, destravava ligações gratuitas de longa distância. Um dos entusiastas usava o apelido "Captain Crunch", em homenagem ao cereal. Dois jovens que vendiam aparelhos para isso, chamados *blue boxes*, se chamavam Steve Jobs e Steve Wozniak. Anos depois, fundariam a Apple.

O Worm de Morris (1988). Um estudante de pós-graduação, Robert Morris, soltou na internet, então minúscula, um programa que se copiava de máquina em máquina. Um erro no código fez ele se replicar rápido demais, travando milhares de computadores. Foi o primeiro grande incidente da internet e levou à criação do primeiro CERT, os times de resposta a emergências que existem até hoje.

Kevin Mitnick (anos 1990). O hacker mais famoso da época ficou conhecido não por códigos geniais, mas por **engenharia social**: ele ligava para empresas se passando por funcionário e convencia as pessoas a entregarem senhas. Preso e depois solto, virou consultor de segurança. Sua história mostrou que o elo mais fraco quase sempre é humano.



Da brincadeira com telefones ao mercado bilionário de hoje.

Stuxnet (2010). Um programa malicioso extraordinariamente sofisticado, descoberto em 2010, sabotou centrífugas de enriquecimento de urânio no Irã, fazendo-as se autodestruir enquanto mostravam aos operadores que estava tudo normal. Amplamente atribuído a uma operação entre Estados Unidos e Israel, marcou a era em que ataques digitais causam efeitos no mundo físico e viram arma de Estado.

A era dos vazamentos (2013 em diante). Vazamentos gigantes (Yahoo, Equifax, e tantos outros) expuseram bilhões de contas e tornaram a segurança uma preocupação de diretoria, não só de técnicos. Nasceu o mercado que existe hoje.

O campo de trabalho hoje

Segurança deixou de ser um nicho e virou uma indústria com falta crônica de profissionais. As principais frentes:

- **Red Team / Pentest**: os que atacam, com autorização, para encontrar falhas antes dos criminosos. É o foco deste livro.
- **Blue Team**: os que defendem, monitoram e respondem a incidentes.
- **Purple Team**: a colaboração entre os dois.
- **AppSec**: segurança dentro do desenvolvimento de software.
- **Forense e resposta a incidentes**: os que investigam depois do ataque.
- **Pesquisa e caça a bugs**: os que encontram falhas em produtos, por conta própria (bug bounty) ou em laboratório.

> **BUG BOUNTY: SER PAGO PARA ACHAR FALHAS**
>
> Muitas empresas mantêm programas de **recompensa por bugs** (*bug bounty*), em plataformas como HackerOne e Bugcrowd. Elas autorizam qualquer pessoa a procurar falhas nos seus sistemas, dentro de regras claras (o *escopo*), e pagam por cada falha válida reportada. É a forma legal de "hackear de verdade" sendo iniciante, e muita gente vive disso. A palavra-chave continua sendo a mesma: **autorização**, aqui dada publicamente pelas regras do programa.

O que a lei diz, no Brasil

- **Invasão de dispositivo informático** (art. 154-A do Código Penal): invadir dispositivo alheio, conectado ou não à rede, para obter,

adulterar ou destruir dados sem autorização do dono, é crime, com pena aumentada se há prejuízo ou vazamento.

- **Interrupção de serviço** e **falsificação** têm artigos próprios.
- A **LGPD** responsabiliza quem trata dados pessoais de forma indevida.

A existência dessas leis é, para o profissional, uma boa notícia: é o que torna o trabalho de quem defende valioso e necessário.

PARA FIXAR

- [ ] Conheço marcos como os phreakers, o Worm de Morris e o Stuxnet.
- [ ] Sei as principais frentes de trabalho da área.
- [ ] Entendi o que é bug bounty e por que é a porta de entrada legal.

PARTE 2

O laboratório

CAPÍTULO 3

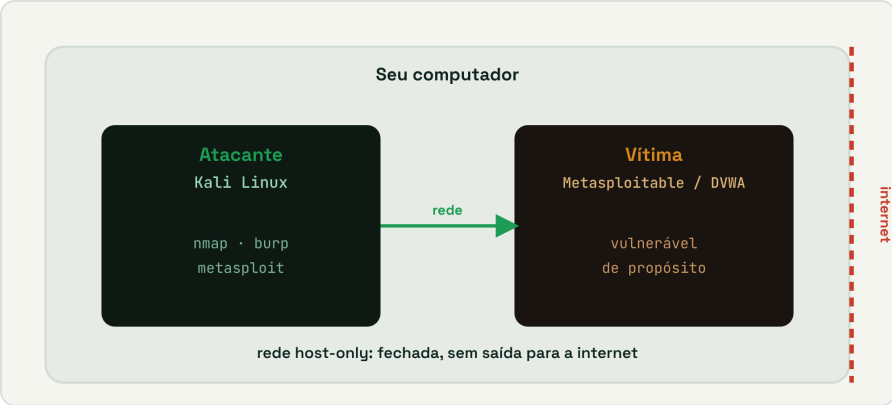
Montando seu laboratório

Você não aprende a dirigir no trânsito da hora do rush, e não aprende hacking em sistemas de verdade. Aprende num **laboratório**: um ambiente isolado, montado no seu próprio computador, onde você pode atacar, quebrar, reinstalar e tentar de novo sem risco para ninguém, e sem risco legal para você.

Este é o capítulo mais importante do livro para começar de fato. Sem laboratório, todo o resto é teoria.

A ideia: máquinas virtuais

Uma **máquina virtual** (VM) é um computador inteiro simulado por software dentro do seu. Você pode ter, na mesma máquina física, uma "máquina atacante" e uma "máquina vítima", conversando entre si numa rede fechada que não toca na internet nem na sua rede de casa.



Tudo acontece dentro do seu computador, numa rede isolada.

As vantagens:

- **Isolamento**: o que acontece no laboratório fica no laboratório.
- **Instantâneos** (*snapshots*): você tira uma "foto" da VM antes de atacar e, se quebrar tudo, volta ao estado anterior em segundos.
- **Descartável**: errou feio? Apaga e cria de novo.

O que instalar

1. Um virtualizador. O programa que roda as VMs. O **VirtualBox** é gratuito e serve para começar. O VMware também é uma opção.

2. A máquina atacante. A distribuição Linux mais usada para segurança é o **Kali Linux**, que já vem com centenas de ferramentas instaladas. Uma alternativa é o **Parrot OS**. Baixe sempre do site oficial e confira a assinatura do arquivo.

3. As máquinas vítimas. Sistemas feitos **de propósito** para serem vulneráveis, para você praticar:

- **Metasploitable**: um Linux cheio de falhas clássicas de rede e serviços.
- **OWASP Juice Shop** e **DVWA** (*Damn Vulnerable Web Application*): aplicações web vulneráveis para treinar ataques a sites.
- **VulnHub**: um catálogo gratuito de VMs vulneráveis, cada uma como um desafio.

> **SE O SEU PC FOR FRACO**
>
> Máquinas virtuais pedem memória. Se o seu computador não aguentar, use as plataformas **na nuvem**: TryHackMe e Hack The Box rodam os laboratórios nos servidores deles, e você acessa pelo navegador ou por uma conexão VPN. O PortSwigger Web Security Academy é totalmente pelo navegador e é gratuito.

A regra de ouro da rede do laboratório

Configure a rede das VMs no modo **"host-only"** ou **"rede interna"**. Isso cria uma rede fechada entre as suas VMs, sem saída para a internet nem para a sua rede doméstica. Assim, nenhuma ferramenta de ataque "escapa" para onde não deve, e você nunca ataca, por engano, um alvo real.

> **NUNCA APONTE FERRAMENTAS DE ATAQUE PARA FORA DO LABORATÓRIO**
>
> Rodar um scanner ou um ataque contra um endereço na internet, mesmo "só para testar", pode ser interpretado como tentativa de invasão e é rastreável até você. Dentro do laboratório, tudo é permitido. Fora dele, nada é, a menos que você tenha autorização por escrito.

Onde praticar de graça e com segurança

Plataforma	O que oferece
TryHackMe	Trilhas guiadas para iniciantes, do zero ao avançado
Hack The Box	Máquinas-desafio, mais puxado, com uma versão “Academy” didática
PortSwigger Web Security Academy	O melhor material gratuito de segurança web, com laboratórios no navegador
OverTheWire	Jogos de linha de comando (o “Bandit” é o começo perfeito)
picoCTF	Competições e desafios para iniciantes, mantidos por uma universidade
VulnHub	VMs vulneráveis para baixar e atacar no seu laboratório

LABORATÓRIO DO CAPÍTULO

- [ ] Instalei um virtualizador (VirtualBox).
- [ ] Criei a máquina atacante (Kali) e tirei um snapshot.
- [ ] Subi uma máquina vítima (Metasploitable ou DVWA).
- [ ] Configurei a rede como host-only e confirmei que as VMs se enxergam.
- [ ] Criei conta no TryHackMe ou no PortSwigger Academy.

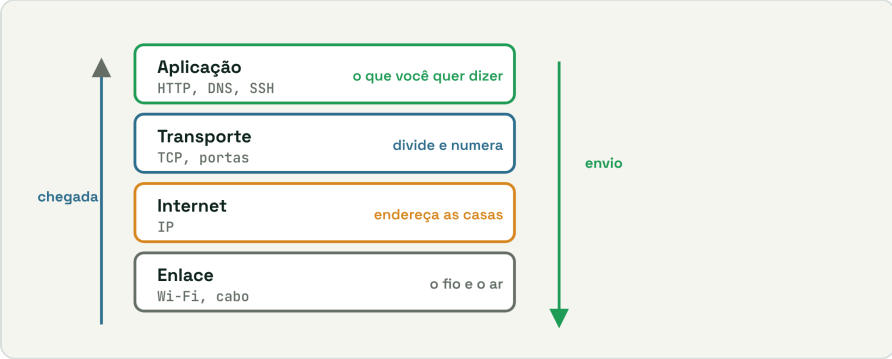
CAPÍTULO 4

As bases: redes e sistemas

Não dá para atacar nem defender o que você não entende. Este capítulo é a base técnica mínima: como os computadores conversam e como os sistemas se organizam. Quem pula esta parte fica preso a copiar comandos sem saber o que eles fazem.

Como dois computadores conversam

Quando o seu navegador abre um site, acontece uma conversa em camadas, como uma carta dentro de envelopes dentro de malotes. O modelo mental mais útil é o **TCP/IP**, com quatro camadas:



A mensagem desce as camadas na origem e sobe no destino. Cada camada tem seu endereço e seu papel.

- **Endereço IP**: o "endereço da casa" de cada máquina na rede, como `192.168.1.10`.

- **Porta**: a “porta de entrada” de um serviço específico naquela máquina. Um servidor pode ter o site na porta 443, o e-mail em outra, o acesso remoto em outra. É como apartamentos num prédio: um IP, muitas portas.
- **Protocolo**: a língua que cada serviço fala. HTTP e HTTPS para sites, SSH para acesso remoto, DNS para traduzir nomes em endereços.

Entender portas e serviços é metade do reconhecimento de rede: atacar começa por descobrir **quais portas estão abertas** e **qual serviço** atende em cada uma, porque cada serviço é uma possível porta de entrada.

Portas e serviços que você vai reencontrar sempre

Porta	Serviço	Para que serve
22	SSH	Acesso remoto seguro ao terminal
80	HTTP	Sites (sem criptografia)
443	HTTPS	Sites (com criptografia)
53	DNS	Tradução de nomes em endereços IP
445	SMB	Compartilhamento de arquivos (Windows)
3306	MySQL	Banco de dados
3389	RDP	Área de trabalho remota (Windows)

O terminal é a sua ferramenta principal

Filmes mostram interfaces coloridas; o trabalho real é no **terminal**, a tela de texto onde você digita comandos. Vale a pena ganhar fluência em Linux: navegar por pastas, ver e editar arquivos, entender permissões (quem pode ler, escrever e executar o quê), e encadear comandos.

O jogo **Bandit**, do OverTheWire, ensina exatamente isso, nível por nível, de forma divertida. É a melhor primeira missão para quem está começando.

Usuários, permissões e privilégios

Todo sistema separa o que cada usuário pode fazer. Existe um usuário comum, com poderes limitados, e um usuário administrador (chamado **root** no Linux, **Administrator** no Windows), que pode tudo.

Boa parte dos ataques tem duas etapas que vêm desse conceito:

1. **Conseguir um primeiro acesso**, geralmente como usuário comum.
2. **Escalar privilégios**: a partir desse acesso limitado, explorar uma falha para virar administrador e controlar a máquina inteira.

Esse padrão, “entrar e depois subir”, aparece em quase todos os capítulos seguintes. Guarde-o.

Um pouco de redes na prática

No seu laboratório, vale brincar com comandos básicos antes de partir para ferramentas de ataque:

- `ip a` (ou `ifconfig`): ver o endereço da sua máquina.
- `ping`: testar se outra máquina responde.
- `nslookup` ou `dig`: perguntar ao DNS o endereço de um nome.

Entender o que cada um mostra já coloca você à frente de quem só decorou ferramentas.

LABORATÓRIO DO CAPÍTULO

- [ ] Descobri o endereço IP da minha máquina atacante e da vítima.
- [ ] Fiz um ping de uma para a outra e vi que se enxergam.
- [ ] Comecei o jogo Bandit do OverTheWire (pelo menos até o nível 5).
- [ ] Sei explicar a diferença entre IP, porta e protocolo.

PARTE 3

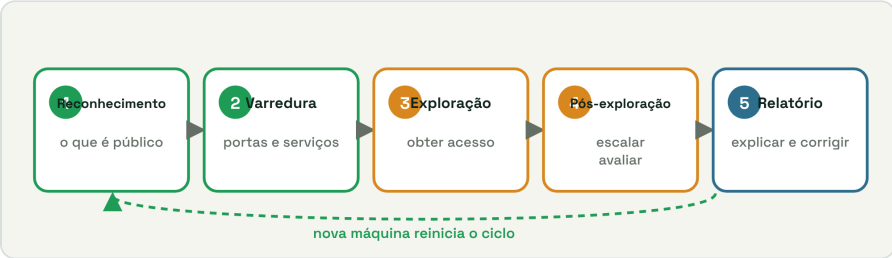
As fases de um teste

CAPÍTULO 5

As cinco fases de um teste

Um teste de invasão profissional não é sair atacando. É um processo com fases bem definidas, o mesmo que você vai seguir nos laboratórios deste livro. Entender o mapa inteiro antes de mergulhar em cada parte evita o erro mais comum do iniciante: pular direto para a exploração sem ter feito o reconhecimento.

O mapa



As fases seguem uma ordem, e a pós-exploração costuma reiniciar o ciclo numa máquina nova.

1. Reconhecimento. Juntar informação sobre o alvo. Numa fase inicial, sem nem tocar no sistema: o que é público, quais domínios, quais tecnologias, quais nomes de funcionários. É a fase mais subestimada e, muitas vezes, a mais decisiva.

2. Varredura (*scanning*). Agora tocando o sistema: descobrir máquinas ativas, portas abertas, serviços rodando e suas versões. É o mapa detalhado da superfície de ataque.

3. Exploração. Usar uma falha encontrada para conseguir acesso. É a fase que todo mundo imagina quando pensa em hacking, e normalmente a mais curta.

4. Pós-exploração. O que você faz depois de entrar: entender onde está, escalar privilégios, ver o que dá para alcançar a partir dali. Num teste real, é aqui que se mede o impacto de verdade.

5. Relatório. A fase que paga as contas. De nada serve achar falhas se você não as explica de forma que a empresa consiga corrigir. Um bom pentester é, acima de tudo, um bom redator.

Por que a ordem importa

Iniciantes querem pular para a fase 3. Mas é o reconhecimento e a varredura que **dizem onde atacar**. Atacar sem isso é tentar arrombar portas ao acaso num prédio enorme, quando havia uma janela aberta no térreo que o reconhecimento teria mostrado.

Uma regra prática do ofício: **se você está travado na exploração, volte ao reconhecimento**. Quase sempre existe uma informação que você não coletou, uma porta que não viu, uma versão de software que não anotou.

Documentar desde o primeiro minuto

Profissionais anotam tudo enquanto trabalham: cada comando, cada resultado, cada ideia. Por três motivos:

- O **relatório** final sai desses registros.
- Quando algo quebra, você precisa saber o que fez para chegar ali.
- Num teste real, você pode precisar **provar** exatamente o que fez e quando.

Comece esse hábito já nos laboratórios: um documento por máquina, com o que você tentou e o que descobriu. Ferramentas como CherryTree e Obsidian são populares para isso, mas um arquivo de texto já serve.

Escopo e autorização, de novo

Num teste profissional, antes de tudo vem um documento que define o **escopo**: exatamente quais sistemas podem ser testados, quais não podem, em que horários, com que limites. Sair do escopo, mesmo sem querer, é falta grave.

Nos seus laboratórios, o escopo é simples: **só as máquinas do laboratório**. Mas pegar o hábito de perguntar "isto está no escopo?" desde já é o que te torna contratável depois.

PARA FIXAR

- [ ] Sei nomear as cinco fases na ordem.
- [ ] Entendi por que reconhecimento vem antes de exploração.
- [ ] Comecei um documento de anotações para os meus laboratórios.

CAPÍTULO 6

Reconhecimento e varredura

O reconhecimento é a arte de saber o máximo sobre o alvo antes de encostar nele a sério. É a fase que separa o profissional do amador, e a boa notícia é que boa parte dela usa só informação pública, sendo perfeita para treinar de forma totalmente legal.

Caso real: a importância de olhar o óbvio

Em muitos testes reais e programas de bug bounty, as falhas mais sérias não começam com um ataque sofisticado, mas com alguém reparando num detalhe público que o dono esqueceu: um painel de login exposto, um servidor de teste acessível, uma senha num repositório de código aberto. Já houve vazamentos enormes que começaram com **uma chave de acesso esquecida dentro do código publicado no GitHub**. O reconhecimento é, antes de tudo, prestar atenção.

Reconhecimento passivo: sem tocar no alvo

É a coleta que não gera nenhum tráfego para o sistema do alvo, usando fontes públicas (a mesma **OSINT** de outros contextos):

- **Registros de domínio** (WHOIS) e **DNS**: quais nomes e endereços a organização tem.

- **Buscadores**, usados com filtros espertos (o chamado *Google dorking*): procurar por tipos de arquivo, páginas de login, mensagens de erro expostas.
- **Sites de inteligência**: Shodan (um buscador de dispositivos conectados), certificados digitais públicos, repositórios de código.
- **Redes sociais e o site da empresa**: nomes de funcionários, tecnologias usadas, padrões de e-mail.

Tudo isso é informação que qualquer pessoa pode ver. Praticar reconhecimento passivo sobre uma empresa é, em geral, legal, porque você não acessa nada que não seja público. Ainda assim, use o bom senso e nunca passe disso sem autorização.

Reconhecimento ativo e varredura: tocando o alvo

Aqui você já manda pacotes ao sistema para descobrir o que existe. **No laboratório**, isto é livre; fora dele, já pode configurar tentativa de invasão.

A ferramenta clássica é o **Nmap** (*Network Mapper*). Ela faz, basicamente, três perguntas:



Quais máquinas existem, quais portas estão abertas, qual serviço e versão atende cada uma.

1. **Quais máquinas estão ativas** na rede?
2. **Quais portas estão abertas** em cada máquina?
3. **Qual serviço e qual versão** atende em cada porta aberta?

A terceira pergunta é a mais valiosa. Saber que a porta 80 está aberta diz pouco; saber que ali roda "Apache versão tal" permite procurar se aquela versão específica tem falhas conhecidas. É a ponte entre a varredura e a exploração.

De versão para vulnerabilidade

Toda falha de segurança conhecida recebe um identificador, um **CVE** (*Common Vulnerabilities and Exposures*), como `CVE-2021-44228`. Existem bancos de dados públicos onde você pesquisa por um software e sua versão e vê a lista de falhas conhecidas, com a gravidade de cada uma.

O fluxo do reconhecimento à exploração é, então:

serviço + versão → procurar CVEs daquela versão → ver se há exploração conhecida → testar no laboratório.

ENUMERAR É INSISTIR

No jargão, essa coleta detalhada chama **enumeração**. A regra de ouro: *enumere mais do que você acha necessário*. Aquela porta estranha, aquele compartilhamento de arquivos aberto, aquele usuário que aparece numa mensagem, qualquer um pode ser o caminho. Iniciantes param cedo; profissionais enumeram até o fim.

LABORATÓRIO DO CAPÍTULO

- [ ] Rodei o Nmap contra a máquina vítima do laboratório.
- [ ] Listei as portas abertas e os serviços com versão.
- [ ] Escolhi um serviço e pesquisei se a versão dele tem CVEs conhecidos.
- [ ] Pratiquei reconhecimento passivo numa das trilhas do TryHackMe.

CAPÍTULO 7

Exploração e pós-exploração

Chegou a fase que todo mundo imagina quando pensa em hacking. E aqui vem a primeira surpresa: num trabalho bem feito, a exploração costuma ser a parte **mais rápida e menos dramática**. Se o reconhecimento foi bom, a exploração é quase uma consequência. Este capítulo explica o *porquê* das técnicas, sempre para você praticar em laboratório.

O que é, no fundo, "explorar uma falha"

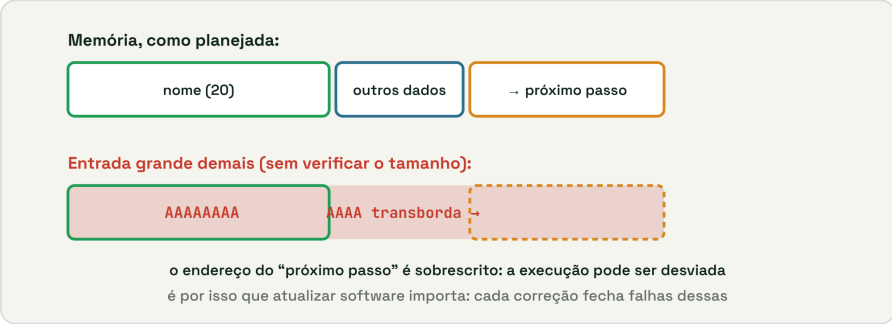
Explorar é fazer um programa executar algo que o autor não previu, aproveitando um erro de lógica ou de programação. As famílias mais comuns de falha:

- **Erros de validação de entrada**: o programa confia em dados que o usuário mandou, e esses dados acabam sendo interpretados como comando. É a raiz da maioria das falhas web (próximo capítulo).
- **Erros de configuração**: senha padrão não trocada, serviço exposto que não devia estar, permissão larga demais. São as falhas mais comuns da vida real, e as mais fáceis.
- **Falhas conhecidas sem correção**: o sistema roda uma versão antiga com um CVE conhecido, e ninguém atualizou.
- **Erros de memória**: programas em certas linguagens podem ser levados a escrever fora do espaço reservado, abrindo caminho

para executar código. É a classe mais técnica, base dos *buffer overflows*.

Um conceito para entender, não para decorar: o buffer overflow

Vale entender a ideia, porque ela explica por que atualizar software importa tanto. Imagine um programa que reserva espaço para um nome de 20 letras. Se ele não verifica o tamanho e você manda 500 letras, as 480 letras extras vão "transbordar" para áreas de memória vizinhas, que guardam, entre outras coisas, o endereço de para onde o programa deve ir em seguida.



Dados demais num espaço pequeno transbordam e sobrescrevem o que vem depois.

Controlando o que transborda, um atacante pode sobrescrever esse endereço e desviar a execução para um código seu. É complexo, os sistemas modernos têm várias proteções contra isso, e dominá-lo leva tempo, mas a lição para todos é simples: **é por isso que software não atualizado é perigoso**. Cada atualização fecha portas dessas.

Frameworks de exploração

Você não escreve cada ataque do zero. Existem **frameworks** que reúnem explorações prontas, de forma organizada, para uso em testes. O mais famoso é o **Metasploit**, que já vem no Kali. Com ele, o fluxo típico em laboratório é:

1. Escolher um módulo de exploração para a falha que você identificou.
2. Configurar o alvo (o IP da máquina vítima do laboratório).
3. Escolher o que acontece ao obter acesso (o *payload*).
4. Executar e, se funcionar, receber uma sessão de acesso à máquina.

Praticar isso contra o **Metasploitable** no seu laboratório é o exercício clássico de iniciante. Você vê, na prática, o ciclo reconhecimento → versão vulnerável → módulo → acesso.

Pós-exploração: entrou, e agora?

Conseguir o primeiro acesso raramente é o fim. A pós-exploração responde: **o que esse acesso realmente permite?**

- **Situar-se**: que máquina é esta, que usuário eu sou, o que tem aqui.
- **Escalar privilégios**: de usuário comum para administrador, explorando uma configuração errada ou uma falha local. É o passo que transforma um acesso limitado em controle total.
- **Mover-se lateralmente**: a partir desta máquina, alcançar outras na mesma rede. Foi assim que vários ataques famosos passaram de um computador comprometido para a rede inteira.

- **Avaliar o impacto**: num teste real, é aqui que se mostra à empresa o que um invasor conseguiria de fato, que dados ele alcançaria.

A DIFERENÇA ENTRE TESTE E CRIME MORA AQUI

Num teste autorizado, a pós-exploração vai só até onde o escopo permite e é toda documentada. Um criminoso faria o mesmo caminho para roubar ou destruir. As ações técnicas podem até coincidir; a autorização, a contenção e a honestidade do relatório são o que define de que lado você está.

LABORATÓRIO DO CAPÍTULO

- [ ] Explorei uma falha do Metasploitable com o Metasploit, no laboratório.
- [ ] Depois de entrar, identifiquei o usuário e o sistema.
- [ ] Pesquisei o conceito de escalonamento de privilégios e tentei um numa máquina do TryHackMe.
- [ ] Entendi, em palavras, o que é um buffer overflow e por que atualizar importa.

PARTE 4

Os grandes domínios

CAPÍTULO 8

Segurança de aplicações web

Se você for escolher **uma** área para se especializar como iniciante, que seja esta. A maior parte do que existe hoje é uma aplicação web, os bug bounties são quase todos de web, e você pode praticar de graça e com segurança no navegador, sem nem montar laboratório. É o domínio com o melhor retorno para o tempo investido.

Por que a web vaza tanto

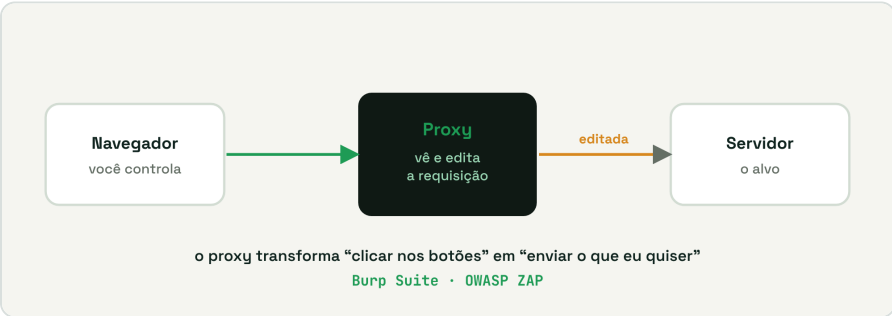
Toda aplicação web tem duas partes: o que roda no seu navegador (*frontend*) e o que roda no servidor (*backend*). Entre as duas, viajam **requisições**. O erro central que causa quase todas as falhas web é um só: **o servidor confiar em dados que vieram do navegador**.

O navegador está sob controle do usuário. Qualquer coisa que ele envia pode ter sido alterada. Quando o servidor trata esses dados como confiáveis, ou os mistura com seus próprios comandos, abre-se a falha.

A ferramenta essencial: o proxy

A ferramenta que define o trabalho de segurança web é um **proxy de interceptação**, como o **Burp Suite** (tem versão gratuita) ou o **OWASP ZAP**. Ele fica entre o seu navegador e o servidor e deixa

você **ver e modificar** cada requisição antes que ela chegue ao destino.



O proxy mostra e deixa editar o que o navegador manda ao servidor.

É o proxy que transforma “clicar nos botões da tela” em “mandar ao servidor exatamente o que eu quiser”. Dominá-lo é o primeiro passo prático da segurança web.

As falhas clássicas, e por que acontecem

O projeto **OWASP** mantém o **Top 10**, a lista das categorias de falha web mais críticas, revisada periodicamente. Vale conhecer as principais pela lógica, não de cor:

- **Injeção (SQL Injection e outras)**: o dado que o usuário manda é interpretado como comando pelo banco de dados. Se um campo de busca é inserido direto numa consulta ao banco, um texto esperto pode mudar a consulta e devolver dados que não deveriam aparecer. A causa e a correção são as mesmas: **separar dados de comandos**.
- **Quebra de controle de acesso**: trocar `id=1032` por `id=1033` e ver o dado de outro usuário, ou acessar uma página de

administração sem ser admin. O servidor esqueceu de verificar **se você tem permissão** para aquilo.

- **XSS (Cross-Site Scripting)**: o site exibe, para outros usuários, um texto que um atacante enviou, e esse texto contém código que roda no navegador das vítimas. De novo: dado tratado como código.
- **Configuração insegura**: páginas de administração abertas, mensagens de erro que revelam demais, senhas padrão.
- **Componentes desatualizados**: a aplicação usa uma biblioteca com falha conhecida.

Repare no padrão que atravessa quase tudo: **misturar dado com comando** e **não verificar permissão**. Entendido o padrão, cada falha específica vira um caso dele.

Onde praticar web (de graça e no navegador)

- **PortSwigger Web Security Academy**: a melhor referência, feita pelos criadores do Burp Suite. Cada falha tem explicação e laboratórios interativos. Gratuito.
- **OWASP Juice Shop**: uma loja virtual propositalmente cheia de falhas, com dezenas de desafios.
- **DVWA**: aplicação vulnerável clássica, com níveis de dificuldade que mostram como cada defesa muda o ataque.

LEIA AS DEFESAS JUNTO COM OS ATAQUES

Em cada falha que estudar, estude também a correção. No DVWA, por exemplo, dá para ver o mesmo ataque falhar conforme o nível de segurança sobe, e ler o código que faz a diferença. Isso te ensina os dois lados de uma vez, e é o que um empregador quer.

LABORATÓRIO DO CAPÍTULO

- ☐ Configurei o Burp Suite (ou o ZAP) com o navegador e interceptei uma requisição.
- ☐ Fiz os primeiros laboratórios de SQL Injection e de controle de acesso no PortSwigger Academy.
- ☐ Resolvi pelo menos três desafios do OWASP Juice Shop.
- ☐ Em cada um, li qual era a correção da falha.

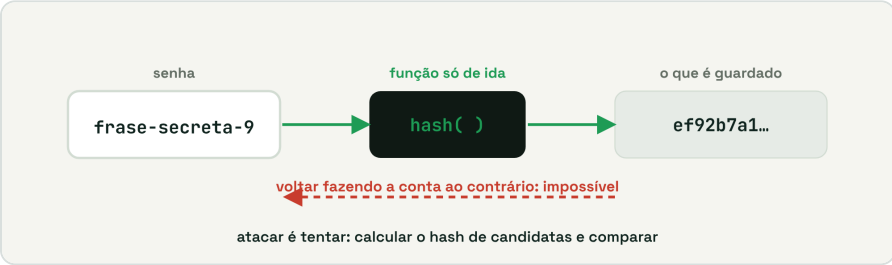
CAPÍTULO 9

Senhas e autenticação

Senhas são a fechadura mais usada do mundo digital e, de longe, a mais frágil. Entender como elas são atacadas é entender por que tudo o que o livro de OPSEC prega (senha única, longa, com segundo fator) é tão importante. Aqui o foco é **como** se quebra uma senha, para você testar a sua própria resistência em laboratório.

Como as senhas são guardadas (ou deveriam ser)

Um sistema bem-feito **nunca** guarda a sua senha como você a digitou. Ele guarda um **hash**: o resultado de passar a senha por uma função matemática que só vai num sentido. De `senha123` sai algo como `ef92b...`, e é impossível voltar do hash para a senha fazendo a conta ao contrário.



A senha vira um hash só de ida. Conferir é refazer o caminho e comparar.

Quando você faz login, o sistema calcula o hash do que você digitou e compara com o que está guardado. Se baterem, você entra. Assim, mesmo que o banco de dados vaze, as senhas não aparecem em texto claro. É a teoria. Na prática, muitos sistemas guardam mal, e é aí que mora o problema.

Como as senhas caem

Já que não dá para "desfazer" um hash, o ataque é por **tentativa**: pega-se uma senha candidata, calcula-se o hash dela e compara-se com o hash vazado. Se bater, achou a senha. As estratégias:

- **Ataque de dicionário**: testar listas de senhas comuns e já vazadas. A lista `rockyou`, com milhões de senhas de um vazamento antigo, é o ponto de partida clássico. É assustadoramente eficaz, porque as pessoas repetem as mesmas senhas.
- **Força bruta**: testar todas as combinações possíveis. Só funciona para senhas curtas; para senhas longas, levaria mais tempo que a idade do universo.
- **Regras e mutações**: pegar o dicionário e aplicar as trocas que as pessoas fazem (`a` por `@`, acrescentar `123` ou o ano no fim). Como esses hábitos são previsíveis, isso quebra muita senha "esperta".
- **Tabelas pré-calculadas** (*rainbow tables*): hashes já calculados para bilhões de senhas, trocando tempo por espaço em disco.

As ferramentas clássicas para isso, usadas em laboratório e em testes autorizados, são o **Hashcat** e o **John the Ripper**.

A defesa que muda o jogo: o “sal”

Por que então os vazamentos não entregam todas as senhas na hora? Porque sistemas bem-feitos usam um **sal** (*salt*): um valor aleatório diferente para cada usuário, misturado à senha antes do hash. Resultado: duas pessoas com a mesma senha têm hashes diferentes, e as tabelas pré-calculadas deixam de funcionar, porque o atacante teria que refazer tudo para cada sal.

Some a isso funções de hash propositalmente **lentas** (como bcrypt, scrypt e Argon2), feitas para que cada tentativa custe tempo, e um ataque que levaria horas passa a levar séculos. Entender isso explica, do lado de dentro, por que a escolha da função de hash é uma decisão de segurança séria.

O elo mais forte que você controla

Do lado do usuário, o que derrota todos esses ataques é o que o livro de OPSEC já dizia, e agora você entende o mecanismo:

- **Comprimento** vence tudo: cada caractere a mais multiplica o esforço do atacante. Uma frase de cinco palavras aleatórias é impraticável de quebrar.
- **Senha única por serviço**: um vazamento não contamina os outros.
- **Segundo fator**: mesmo com a senha, o atacante não entra.

TESTE A SUA PRÓPRIA SENHA, COM SEGURANÇA

No seu laboratório, você pode criar o hash de uma senha sua e ver quanto tempo o Hashcat leva para quebrá-la, do dicionário à força bruta. É um exercício revelador: senhas "difíceis" como `P@ssw0rd!` caem em segundos, enquanto uma frase longa resiste. Faça isso só com senhas **suas**, criadas para o teste.

LABORATÓRIO DO CAPÍTULO

- [ ] Entendi por que sistemas guardam hashes, e não senhas.
- [ ] Criei o hash de uma senha de teste e quebrei com John ou Hashcat no laboratório.
- [ ] Comparei o tempo entre uma senha curta "esperta" e uma frase longa.
- [ ] Sei explicar o que é sal e por que ele derrota as rainbow tables.

CAPÍTULO 10

Redes sem fio e o fator humano

Dois domínios muito diferentes, unidos por uma coisa: nos dois, a falha quase nunca é a criptografia em si, e quase sempre é como as pessoas a usam.

Redes Wi-Fi

O Wi-Fi é interessante para aprender porque ele acontece pelo ar: qualquer antena por perto "ouve" os pacotes. É por isso que a criptografia da rede é tão importante, e por isso que redes mal configuradas são um alvo didático.

A evolução da proteção. Os padrões de segurança do Wi-Fi contam uma história de falhas e correções:

Padrão	Situação
WEP	Quebrado há mais de 20 anos. Nunca use.
WPA / WPA2	O comum hoje. O WPA2 é sólido, desde que a senha seja forte.
WPA3	O mais novo, corrige fraquezas do WPA2.

Onde a fraqueza mora. No WPA2 doméstico, a segurança depende inteiramente da **senha**. Quando um dispositivo se conecta, há uma troca inicial (o *handshake*) que um atacante por perto pode

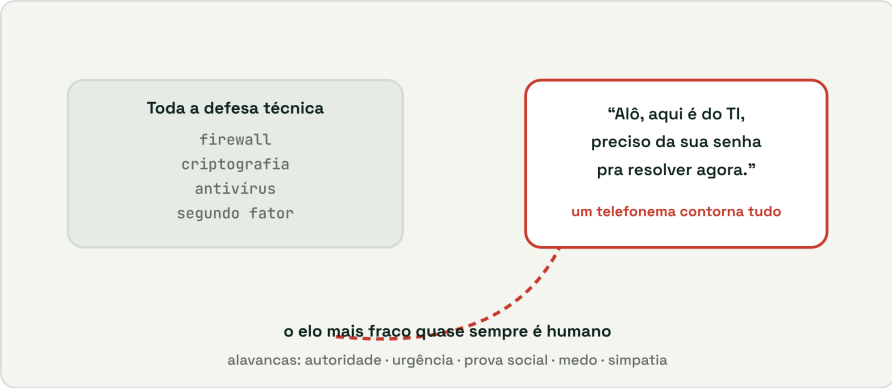
capturar. Com essa captura, ele tenta adivinhar a senha **offline**, com os mesmos ataques de dicionário do capítulo anterior, sem precisar mais da rede. A conclusão prática: o `12345678` do roteador cai rápido; uma senha longa, não.

Como praticar com segurança. Atacar o Wi-Fi do vizinho é crime. Para treinar de verdade e sem risco, monte o alvo: um **roteador antigo seu**, dedicado ao laboratório, com uma senha de teste que você mesmo define. Aí você captura o próprio handshake e tenta quebrar a própria senha. Há também laboratórios no TryHackMe que simulam tudo isso sem hardware.

> **O AR NÃO RESPEITA PAREDES, A LEI SIM**
>
> Capturar tráfego de uma rede que não é sua, ou conectar-se a ela sem permissão, é ilegal, mesmo que a rede esteja "aberta". O fato de o sinal chegar até você não é autorização. Treine só na sua própria rede de laboratório.

Engenharia social: o hack que não precisa de código

Lembra do Kevin Mitnick, do Capítulo 2? A técnica mais eficaz de todas raramente envolve explorar software. Envolve explorar **pessoas**. Por que gastar semanas quebrando uma defesa técnica se dá para ligar para alguém e pedir a senha com uma boa história?



Toda a tecnologia de segurança pode ser contornada por um telefonema convincente.

As alavancas são sempre as mesmas, as mesmas dos golpes do livro de OPSEC: **autoridade** (“aqui é do TI”), **urgência** (“preciso disso agora”), **prova social** (“seu colega já me passou”), **medo** e **simpatia**.

As formas mais comuns num teste autorizado:

- **Phishing**: e-mails ou mensagens que induzem a clicar ou digitar credenciais num site falso.
- **Pretexting**: inventar um papel e uma história para extrair informação.
- **Vishing**: a mesma coisa, por telefone.
- **Baiting**: deixar um pendrive “perdido” no estacionamento, contando com a curiosidade de quem achar.

Em testes profissionais, campanhas de engenharia social são contratadas pelas empresas para medir o preparo dos funcionários, sempre com autorização e com cuidado ético: o objetivo é treinar as pessoas, nunca humilhá-las.

A LINHA ÉTICA AQUI É MAIS FINA

Enganar pessoas, mesmo num teste, exige regras claras e autorização de quem manda. Fora de um teste contratado, aplicar essas técnicas em alguém é manipulação e, dependendo do caso, crime (estelionato, por exemplo). Estude engenharia social principalmente para **reconhecer e defender**.

A defesa, nos dois casos

- **Wi-Fi**: WPA2 ou WPA3 com senha longa, rede de convidados separada, firmware do roteador atualizado.
- **Pessoas**: a única defesa real é a **consciência**. Treinamento, o hábito de verificar por outro canal e uma cultura em que "desconfiar e conferir" não é visto como falta de educação.

LABORATÓRIO DO CAPÍTULO

- [ ] Entendi por que a senha é o ponto fraco do WPA2.
- [ ] (Opcional) Montei um roteador velho como alvo e capturei meu próprio handshake.
- [ ] Fiz uma trilha de engenharia social no TryHackMe.
- [ ] Listei as cinco alavancas psicológicas e como reconhecê-las.

PARTE 5

O outro lado e a carreira

CAPÍTULO 11

Pensar como quem defende

Aprender a atacar tem um objetivo maior: aprender a defender. Cada falha deste livro tem o seu espelho do lado azul. E, no mercado, a maioria das vagas é de defesa. Este capítulo mostra o outro lado do que você aprendeu.

Toda falha tem uma correção

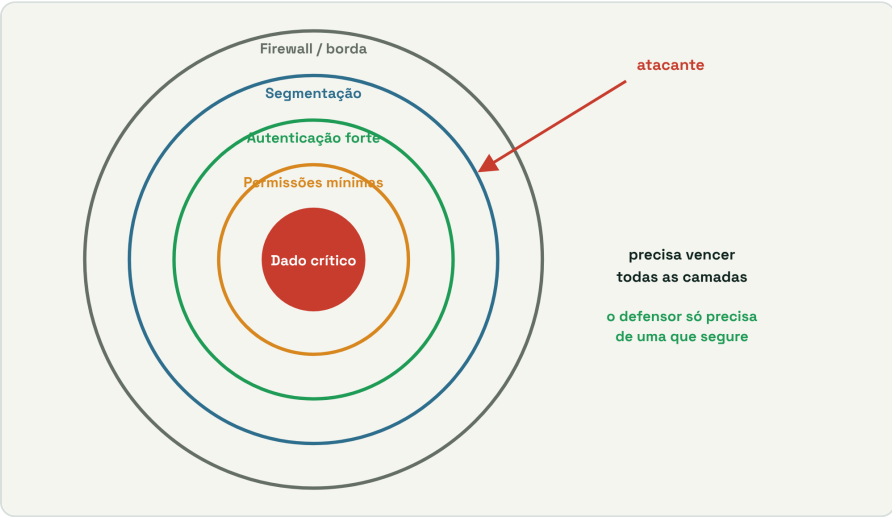
Repasse o livro de trás para a frente, agora pelos olhos de quem protege:

O ataque	A defesa
Versão vulnerável achada na varredura	**Atualizar** e reduzir o que fica exposto
Injeção e XSS na web	**Separar dado de comando**, validar entradas
Trocar `id=1032` por `id=1033`	**Verificar permissão** em cada acesso
Senha quebrada por dicionário	**Hash lento com sal**, senha longa, segundo fator
Handshake de Wi-Fi quebrado	**Senha forte**, WPA3
Phishing bem-sucedido	**Consciência**, segundo fator, verificação por outro canal

Repare que as defesas são poucas e se repetem: **reduzir a superfície exposta, atualizar, validar entradas, verificar permissões, usar bons segredos e treinar pessoas**. Quem entende os ataques entende por que cada uma dessas defesas existe, e é por isso que o melhor defensor sabe atacar.

As camadas da defesa

A boa segurança não aposta numa muralha só, porque toda muralha cai. Ela se organiza em **camadas** (a chamada *defesa em profundidade*): se uma falha, a próxima segura. Firewall, segmentação da rede, monitoramento, backup, permissões mínimas, autenticação forte. O atacante precisa vencer todas; o defensor só precisa que uma segure.



O atacante precisa atravessar tudo. O defensor só precisa que uma camada segure.

O Blue Team e o SOC

Do lado defensivo, muita gente trabalha num **SOC** (*Security Operations Center*): a sala que monitora, 24 horas, os sinais de ataque. As atividades:

- **Monitorar** registros e alertas em busca de atividade suspeita.
- **Responder a incidentes**: conter, investigar e recuperar quando algo acontece.
- **Caçar ameaças** (*threat hunting*): procurar ativamente invasores que passaram pelos alarmes.
- **Forense**: reconstruir o que aconteceu depois de um ataque.

Para treinar o lado azul, há o **Blue Team Labs Online**, as trilhas defensivas do TryHackMe e ferramentas como o **Security Onion**, que você pode montar no laboratório para ver os seus próprios ataques aparecerem nos alertas, o exercício que mais ensina os dois lados de uma vez.

O ciclo que fecha

Ataque e defesa não são times rivais; são um ciclo. O *red team* ataca, o *blue team* aprende com o ataque e fecha a falha, o que obriga o *red team* a achar um caminho novo. Toda a área avança por esse vaivém. Entender os dois lados não é opcional para quem quer ir longe; é o caminho.

LABORATÓRIO DO CAPÍTULO

- [ ] Para cada ataque que pratiquei, escrevi qual seria a defesa.
- [ ] Entendi o conceito de defesa em profundidade.
- [ ] Experimentei uma trilha defensiva (Blue Team) e vi um ataque aparecer nos alertas.

CAPÍTULO 12

Seu caminho a partir daqui

Você chegou ao fim do mapa. Agora vem a parte que livro nenhum faz por você: a prática constante. Este capítulo é o plano para transformar o que leu num caminho real, seja por hobby, seja por profissão.

Os primeiros 90 dias

Um plano concreto, do zero ao ponto de resolver desafios sozinho:

Mês 1 — as bases. Monte o laboratório (Capítulo 3). Ganhe fluência no terminal Linux com o **Bandit** do OverTheWire, do começo ao fim. Faça as trilhas introdutórias do **TryHackMe** ("Pre Security" e "Intro to Cyber Security"). Meta: sentir-se em casa no terminal e com os conceitos de rede.

Mês 2 — a web. Mergulhe no **PortSwigger Web Security Academy**, do começo. Em paralelo, resolva desafios do **OWASP Juice Shop**. A web é onde há mais oportunidade e mais material gratuito. Meta: entender e explorar, em laboratório, as falhas do OWASP Top 10.

Mês 3 — máquinas completas. Comece a atacar máquinas inteiras no **TryHackMe** (trilha "Jr Penetration Tester") e, quando ganhar confiança, no **Hack The Box**. Pratique o ciclo completo: reconhecimento, varredura, exploração, escalonamento. Meta: resolver uma máquina "fácil" do início ao fim sozinho.



Um caminho possível. O importante é a constância, não a velocidade.

Documentar é construir carreira

Desde o primeiro dia, **escreva o que você faz**. Mantenha um blog, um perfil no GitHub, anotações públicas de como resolveu cada desafio (os chamados *write-ups*, respeitando as regras de cada plataforma sobre o que pode ser publicado). Isso faz três coisas:

- **Fixa o aprendizado**: explicar é a melhor forma de entender.
- **Cria portfólio**: num campo onde diploma importa menos que capacidade, seus write-ups e máquinas resolvidas são o seu currículo.
- **Te conecta**: a comunidade de segurança é ativa e acolhedora com quem mostra trabalho honesto.

Certificações, quando fizerem sentido

Não são obrigatórias para aprender, mas ajudam a entrar no mercado. As mais citadas, da base ao avançado:

- **CompTIA Security+**: fundamentos amplos, boa para o currículo inicial.
- **eJPT**: prática e acessível, focada em pentest de entrada.

- **PNPT** e **OSCP**: práticas e respeitadas; a OSCP é um marco exigente, com prova de 24 horas atacando máquinas reais de laboratório.

Comece pela prática gratuita. Pague por certificação quando tiver clareza de que quer seguir na área e de qual frente.

A comunidade e a conduta

A reputação, nesta área, é tudo, e se constrói devagar e se perde rápido. Três princípios que acompanham os melhores profissionais:

1. **Autorização sempre.** É a linha que você nunca cruza. Na dúvida, não teste.
2. **Divulgação responsável.** Achou uma falha num sistema real por acaso? Reporte ao dono de forma privada e dê tempo para corrigir, antes de tornar pública. É o que separa o pesquisador do oportunista.
3. **Generosidade.** A área cresceu com gente compartilhando conhecimento de graça. Retribua.

Uma última palavra

Hacking, no fundo, é a recusa de aceitar que as coisas só funcionam do jeito que te disseram. Essa curiosidade é valiosa demais para ser desperdiçada em coisas pequenas ou ilegais. Aponte-a para aprender, para construir, para defender o que importa. O mundo tem falta de gente que entende como as coisas quebram e escolhe consertá-las.

Agora fecha o livro e vai montar o laboratório. É lá que se aprende de verdade.

O PRIMEIRO PASSO, HOJE

- [ ] Criei conta no TryHackMe e comecei a primeira trilha.
- [ ] Comecei o Bandit do OverTheWire.
- [ ] Abri um documento ou repositório para registrar o que for aprendendo.

Glossário

Bug bounty
Programa em que uma empresa autoriza e paga pela descoberta de falhas nos seus sistemas, dentro de regras públicas.

Buffer overflow
Falha em que dados além do espaço reservado sobrescrevem a memória vizinha, podendo desviar a execução do programa.

CVE
Identificador público e único de uma falha de segurança conhecida (ex.: CVE-2021-44228).

Enumeração
Coleta detalhada de informações sobre um alvo: usuários, serviços, versões, compartilhamentos.

Escopo
Definição do que pode e do que não pode ser testado num teste autorizado.

Escalonamento de privilégios
Passar de um acesso limitado para um de administrador.

Engenharia social
Manipular pessoas para obter informação ou acesso.

Exploit

Código ou técnica que aproveita uma falha para obter um efeito não previsto.

Hash

Resultado de uma função de só ida aplicada a um dado; usado para guardar senhas sem guardá-las em texto claro.

Handshake

Troca inicial entre dois sistemas ao estabelecer uma conexão; no Wi-Fi, pode ser capturada para ataque offline.

Máquina virtual (VM)

Computador simulado por software, usado para montar laboratórios isolados.

Nmap

Ferramenta de varredura de redes: descobre máquinas, portas e serviços.

OSINT

Inteligência de fontes abertas: coleta de informação pública sobre um alvo.

OWASP Top 10

Lista das categorias de falha web mais críticas, mantida pelo projeto OWASP.

Payload

A parte de um ataque que produz o efeito desejado após a exploração.

Pentest

Teste de invasão autorizado, para encontrar falhas antes dos criminosos.

Phishing

Mensagem falsa que induz a vítima a clicar ou entregar credenciais.

Porta

Ponto de entrada de um serviço específico numa máquina (ex.: 443 para HTTPS).

Proxy de interceptação

Ferramenta que fica entre navegador e servidor, deixando ver e editar as requisições (ex.: Burp Suite).

Red team / Blue team

Os que atacam (com autorização) e os que defendem.

Sal (salt)

Valor aleatório misturado à senha antes do hash, para derrotar tabelas pré-calculadas.

SQL Injection

Falha em que dados do usuário são interpretados como comando pelo banco de dados.

Write-up

Relato de como um desafio foi resolvido; serve de estudo e de portfólio.

XSS

Cross-Site Scripting: injetar código que roda no navegador de outros usuários de um site.

Entenda como quebra.
Depois, conserte.